

데이터베이스 시스템 개요

학습목표

1. 데이터베이스 시스템 개요
2. DBMS를 사용하는 데이터 관리
3. DBMS의 발전
4. 데이터베이스 스키마/인스턴스
5. 데이터베이스 언어
6. 데이터베이스 사용자
7. 데이터 독립성
8. 데이터베이스 시스템 아키텍처

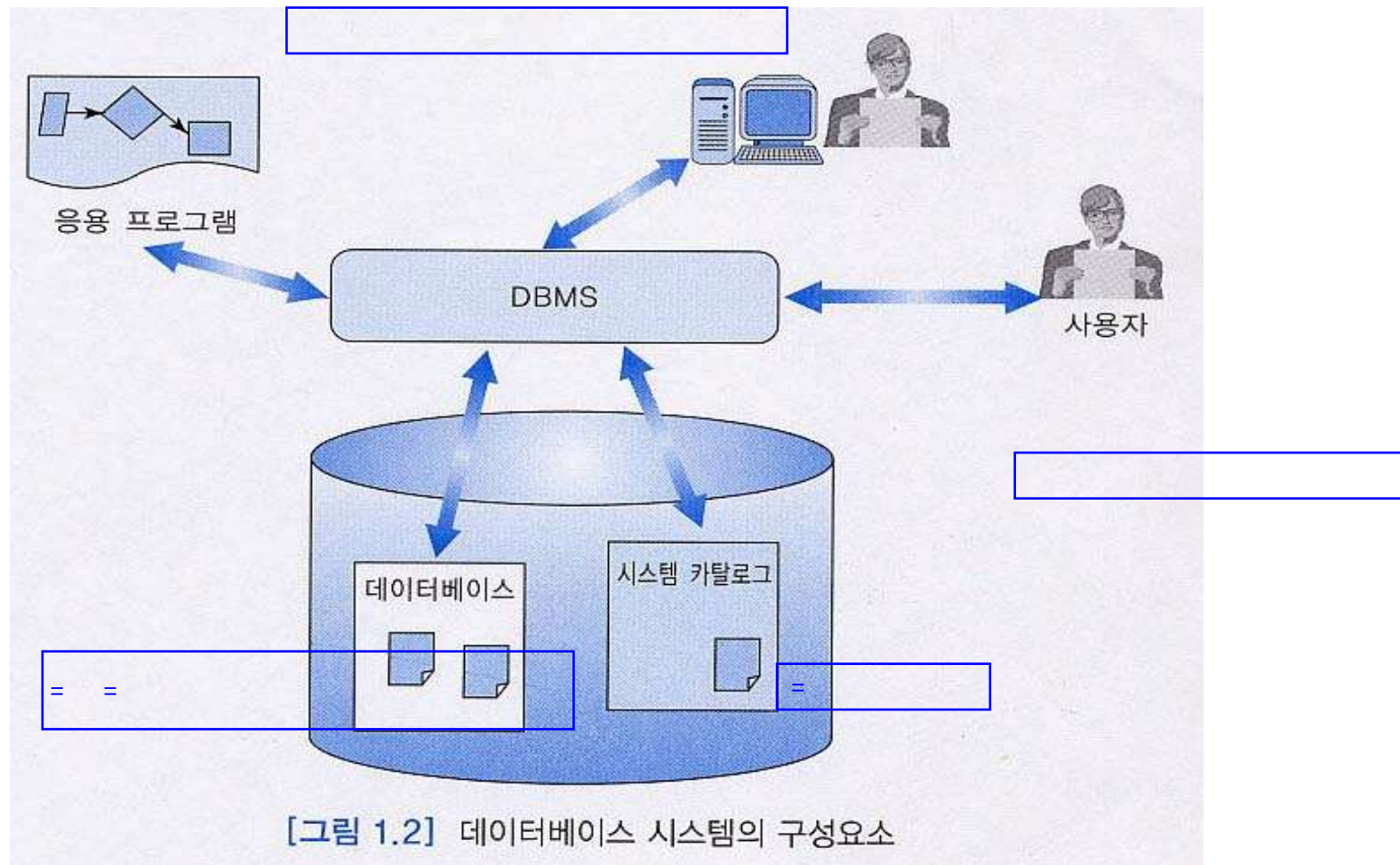
1. 데이터베이스 시스템

- ❑ 정보화 사회(특히 빅 데이터 시대)에서 정보를 수집, 관리, 분석하는데 데이터베이스 관련 기술이 주요하게 활용되고 있음
- ❑ **[용어 정의 및 구분]** 데이터베이스 시스템(Database System)
 - ✓ 데이터베이스(Database)
 - ✓ 데이터베이스 관리 시스템(Database Management System)
 - ✓ 데이터베이스 사용자 (User) 및 응용 프로그램



1.1 데이터베이스 시스템 개요

❑ 데이터베이스 시스템(DBS: Database System)의 구성 요소



1.1 데이터베이스 시스템 개요

❑ 데이터베이스(database) 정의

데이터베이스는 조직체에서 공유해서 사용하는 운영 데이터 (operational data)들을 구조적으로 통합하여 저장한 서로 관련된 데이터의 모임이다.

❑ 데이터베이스의 특징

- ✓ 데이터베이스는 데이터의 대규모 저장소로서, 여러 목적으로 사용자에게 의해 동시에 사용됨 (동시 공유)
- ✓ 모든 데이터가 중복을 최소화하면서 통합됨 (중복 최소화)
- ✓ 데이터베이스는 한 조직체의 운영 데이터뿐만 아니라 그 데이터에 관한 설명(메타데이터(metadata))까지 포함 (자기 기술)

1.1 데이터베이스 시스템 개요

□ 데이터베이스

- ✓ 조직체의 응용 시스템들이 공유해서 사용하는 서로 관련된 운영 데이터들이 구조적으로 통합된 모임
- ✓ 시스템 카탈로그(또는 데이터 사전)와 저장된 데이터베이스로 구분할 수 있음
- ✓ 시스템 카탈로그(system catalog)는 저장된 데이터베이스의 스키마 정보(메타 정보)를 유지

1.1 데이터베이스 시스템 개요

□ 데이터베이스 관리 시스템(DBMS: Database Management System)

- ✓ 데이터베이스와 사용자간의 인터페이스 기능
- ✓ 데이터베이스를 정의하고, 질의 처리를 지원하고, 접근 관리 기능 등의 작업을 수행하는 소프트웨어

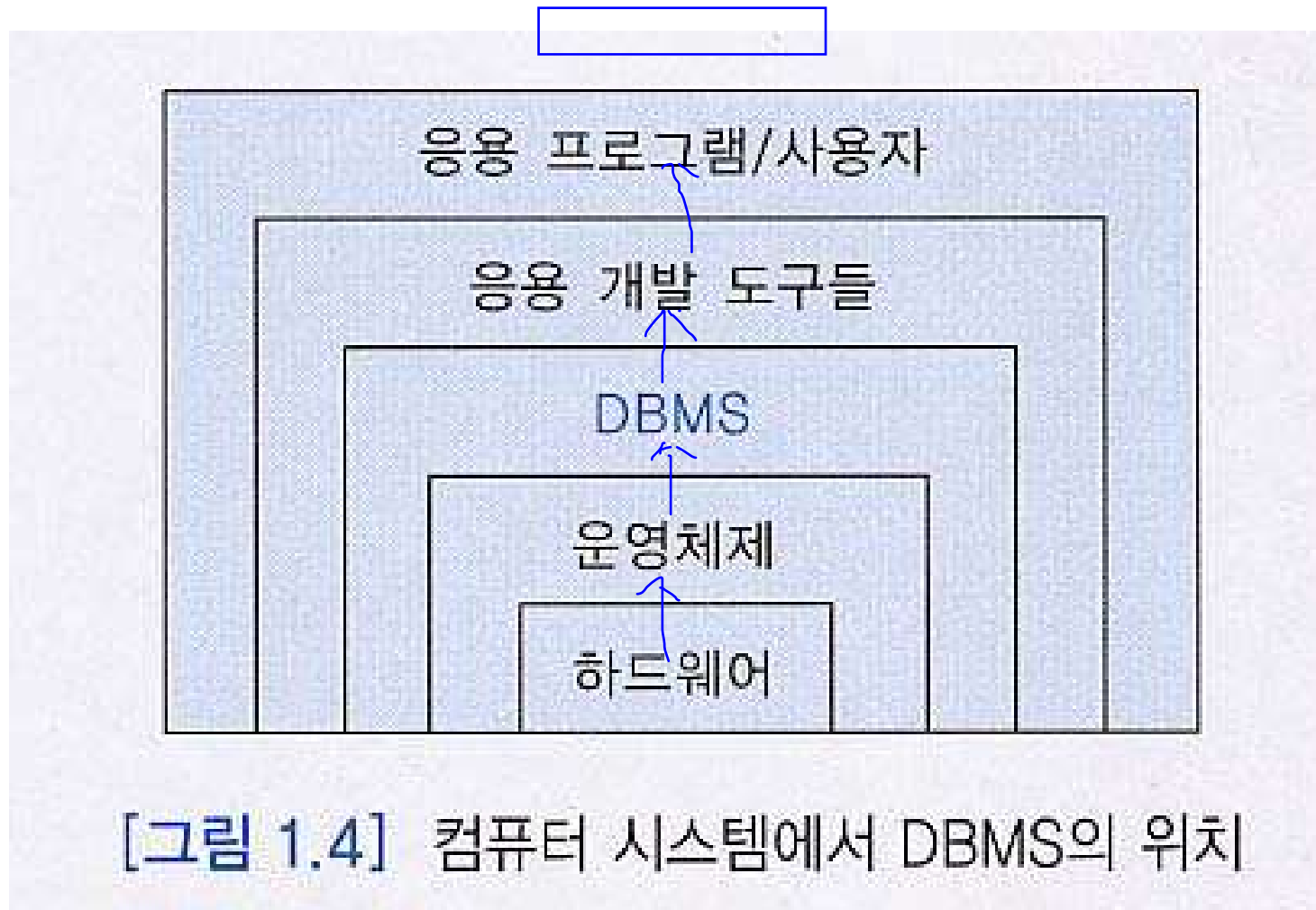


1.1 데이터베이스 시스템 개요

□ DBMS 기능

- ✓ 사용자가 새로운 데이터베이스를 생성하고, 데이터베이스의 구조를 명세할 수 있게 하고(정의 기능), 사용자가 데이터를 효율적으로 질의하고 수정할 수 있도록 하며(조작 기능), 시스템의 고장이나 권한이 없는 사용자로부터 데이터를 안전하게 보호하며(보호 기능), 동시에 여러 사용자가 데이터베이스를 접근하는 것을 제어하는(동시 공유 지원 기능) 소프트웨어 패키지
- ✓ 데이터베이스 언어(Database language)라고 부르는 특별한 프로그래밍 언어를 한 개 이상 제공
- ✓ SQL은 여러 DBMS에서 제공되는 사실상의 표준 데이터베이스 언어

1.1 데이터베이스 시스템 개요



1.1 데이터베이스 시스템 개요

□ 사용자

- ✓ 데이터베이스 사용자는 여러 부류로 나눌 수 있음
- ✓ 뒤에서 상세 구분 설명 예정

□ 하드웨어

- ✓ 데이터베이스(DB)는 디스크와 같은 보조 기억 장치에 저장되며, DBMS에서 원하는 정보를 찾기 위해서는 디스크의 블록들을 주 기억 장치로 읽어 들여야 하며, 계산이나 비교 연산들을 수행하기 위해 중앙 처리 장치(CPU)가 사용됨
- ✓ DBMS 자체도 주 기억 장치에 적재되어 실행되어야 함

1.1 데이터베이스 시스템 개요(계속)

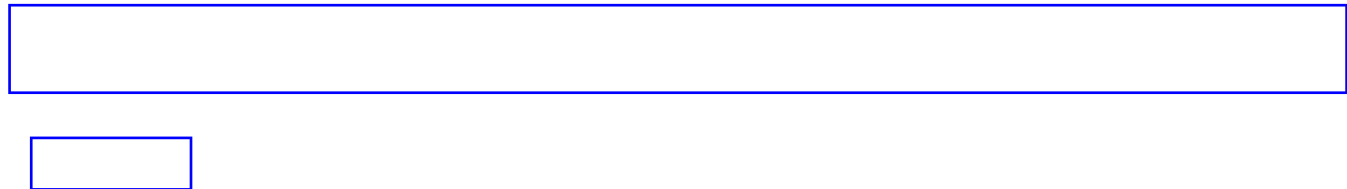
❑ 데이터베이스 시스템의 요구사항(DBMS의 기능적인 주요 특성)

- ✓ 데이터 독립성
- ✓ 효율적인 데이터 접근
- ✓ 데이터에 대한 동시 접근
- ✓ 백업과 회복
- ✓ 쉬운 질의어
- ✓ 다양한 사용자 인터페이스
- ✓ 중복을 줄이거나 제어하며 일관성 유지
- ✓ 데이터 무결성
- ✓ 데이터 보안

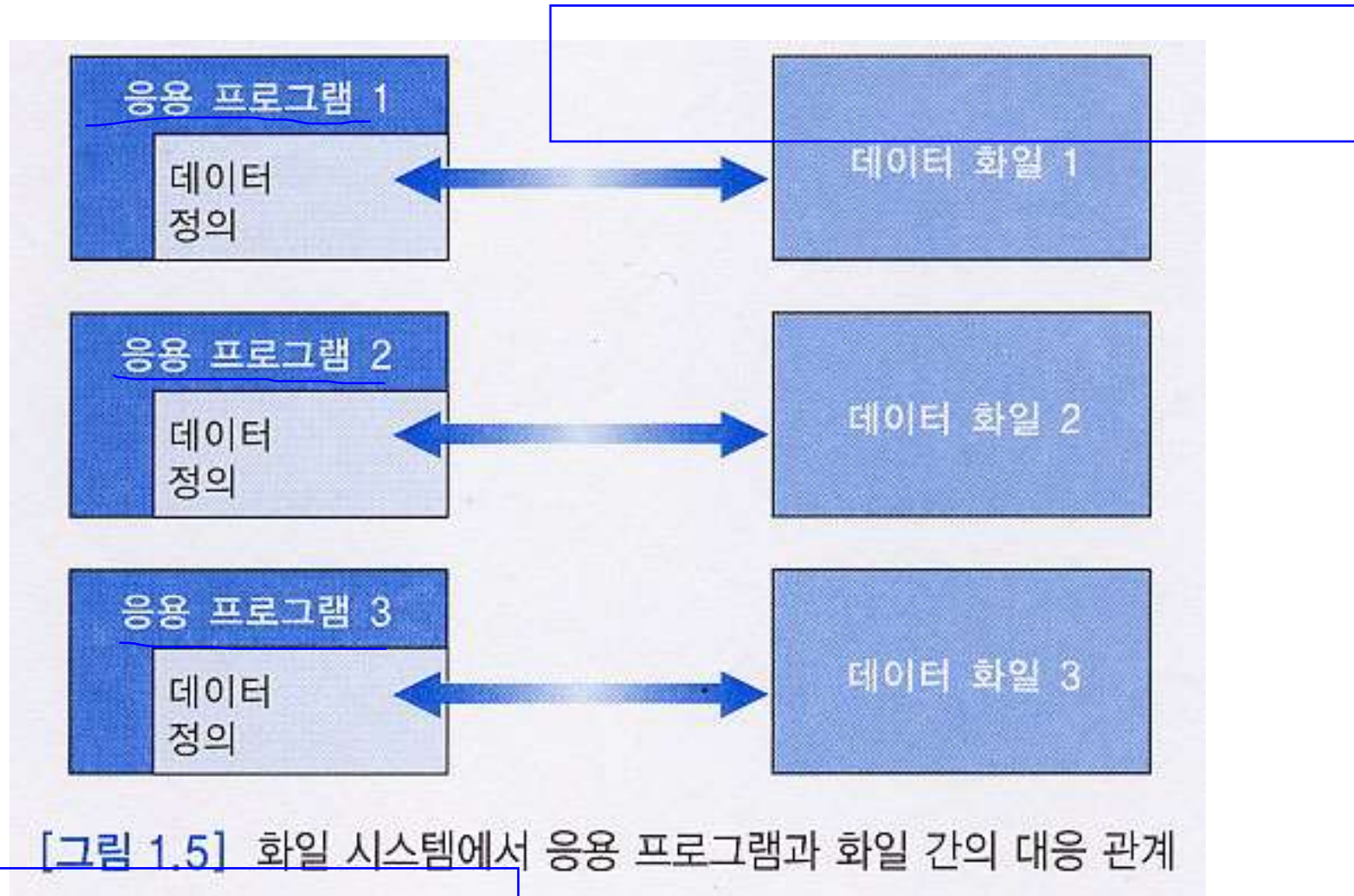
1.2 화일 시스템 vs. DBMS

□ 화일(파일) 시스템을 사용한 기존의 데이터 관리

- ✓ 화일 시스템은 DBMS가 본격적으로 업무에 사용되기 이전에 사용되어 왔음
- ✓ 화일의 기본적인 구성요소는 순차적인 레코드들
- ✓ 한 레코드는 연관된 필드들의 모임
- ✓ 화일을 접근하는 방식이 응용 프로그램 내에 상세하게 표현되므로 데이터에 대한 응용 프로그램의 의존도가 높음



1.2 화일 시스템 vs. DBMS(계속)

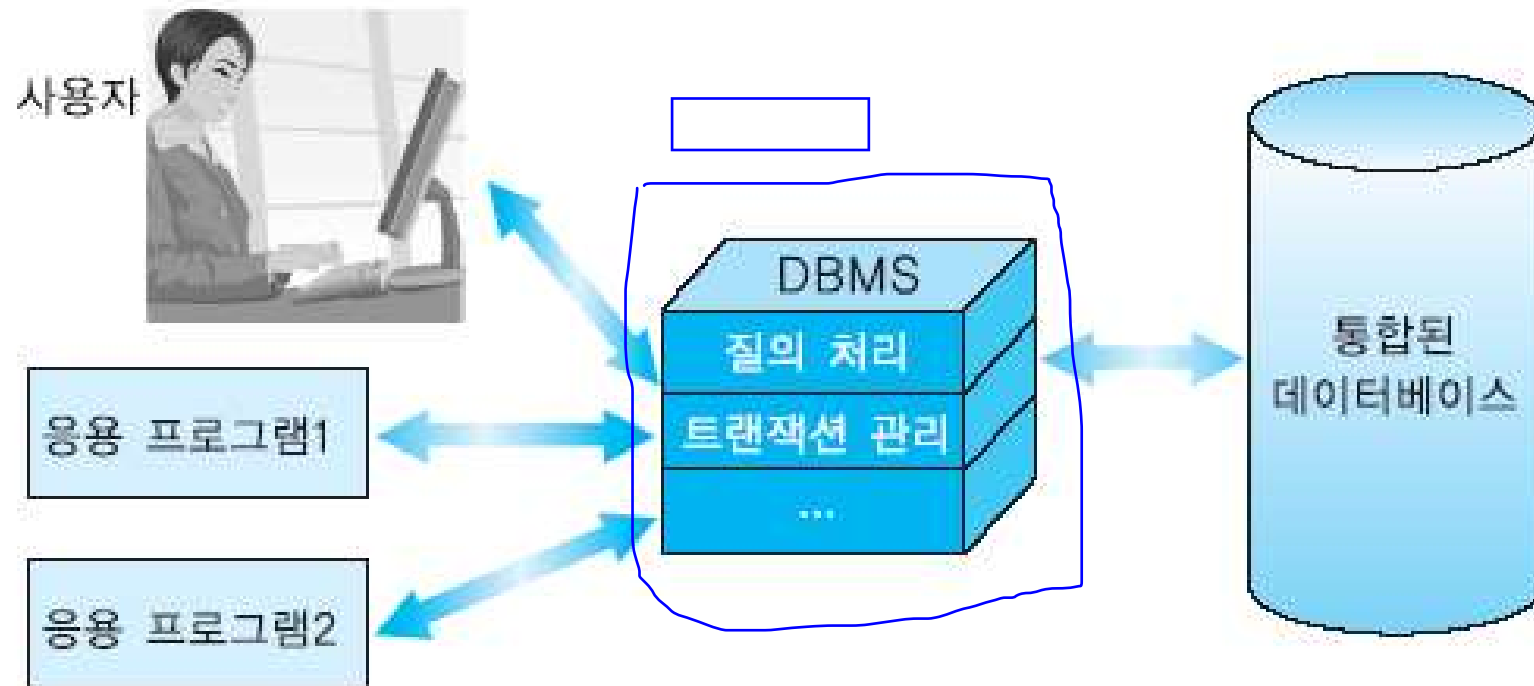


1.2 화일 시스템 vs. DBMS(계속)

□ DBMS를 사용한 데이터베이스 관리

- ✓ 여러 사용자와 응용 프로그램들이 데이터베이스를 공유
 - ✓ 사용자의 질의를 빠르게 수행할 수 있는 인덱스 등의 접근 경로를 DBMS가 자동적으로 선택하여 수행
 - ✓ 권한이 없는 사용자로부터 데이터베이스를 보호
 - ✓ 여러 사용자의 수준에 적합한 다양한 인터페이스를 제공
 - ✓ 데이터 간의 복잡한 관계를 표현하며, 무결성 제약조건을 DBMS가 자동적으로 유지
 - ✓ 시스템이 고장 나면 데이터베이스를 고장 전의 일관된 상태로 회복시킴
 - ✓ 프로그램에 영향을 주지 않으면서 데이터베이스 구조를 변경할 수 있음
- 프로그램-데이터 독립성(program-data independence)

1.2 화일 시스템 vs. DBMS(계속)



[그림 1.8] DBMS를 사용한 데이터베이스 관리

1.2 화일 시스템 vs. DBMS(계속)

□ DBMS 사용의 장점

- ✓ 중복성과 불일치가 감소됨
- ✓ 응용 시스템을 추가 개발하고 유지하는 비용이 감소됨
- ✓ 표준화를 시행하기가 용이
- ✓ 보안이 향상됨
- ✓ 무결성이 향상됨
- ✓ 조직체의 요구사항을 종합적으로 파악할 수 있음
- ✓ 다양한 유형의 고장으로부터 데이터베이스를 회복할 수 있음
- ✓ 데이터베이스의 공유와 동시 접근이 가능함

1.3 DBMS 발전 과정

□ 데이터 모델(data model)

- ✓ 데이터베이스의 구조를 기술하는데 사용되는 개념들의 집합으로, 데이터 구조(데이터 타입과 관계), 이 구조 위에서 동작하는 연산자들, 무결성 제약조건을 포함
- ✓ 사용자에게 내부 저장 방식의 세세한 사항은 숨기면서 데이터에 대한 직관적인 뷰를 제공하는 동시에 이들 간의 사상을 제공 (데이터 추상화)



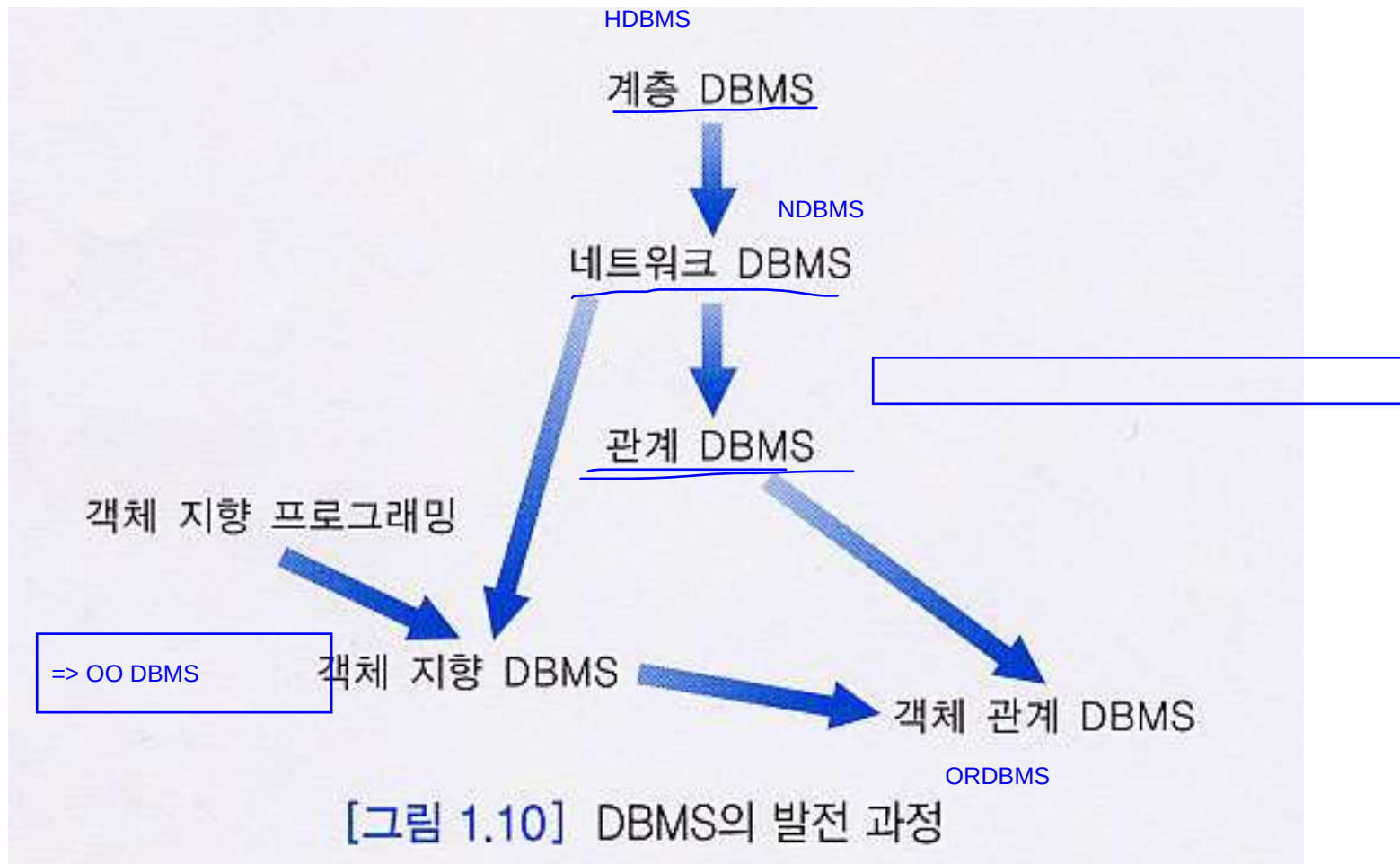
[그림 1.9] 관계 데이터 모델에서의 실세계 표현

1.3 DBMS 발전 과정(계속)

□ 데이터 모델의 분류

- ✓ **고수준** 또는 개념적 데이터 모델(conceptual data model)
 - 사람이 인식하는 것과 유사하게 데이터베이스의 전체적인 논리적 구조를 명시
 - 예: **개체-관계**(ER: Entity-Relationship) 데이터 모델과 객체 지향 데이터 모델
- ✓ **표현(구현)** 데이터 모델(representation(implementation) data model)
 - 최종 사용자가 이해하는 개념이면서 컴퓨터 내에서 데이터가 조직되는 방식과 멀리 떨어져 있지는 않음
 - 예: 계층 데이터 모델(hierarchical data model), 네트워크 데이터 모델(network data model), **관계 데이터 모델**(relational data model)
- ✓ **저수준** 또는 물리적인 데이터 모델(physical data model)
 - 데이터베이스에 데이터가 어떻게 저장되는가를 기술
 - 예: Unifying, ISAM, VSAM 등

1.3 DBMS 발전 과정(계속)



1.3 DBMS 발전 과정(계속)

□ 관계 DBMS

- ✓ 1970년에 E.F. Codd가 IBM 연구소에서 관계 데이터 모델을 제안
- ✓ 미국 IBM 연구소에서 진행된 System R과 캘리포니아 버클리대에서 진행된 Ingres 프로젝트
- ✓ 장점
 - 모델이 간단하여 이해하기 쉬움 (수학적 개념으로부터 출발)
 - 사용자는 자신이 원하는 것(what)만 명시하고, 데이터가 어디에 있는지, 어떻게 접근해야 하는지는 DBMS가 결정
- ✓ 예: MS SQL Server, Oracle, Sybase, DB2, MySQL, MS Access 등

1.3 DBMS 발전 과정(계속)

STUDENT

NAME	ST_NO	ADDRESS	DEPT	GRADE
송치윤	52015	사당동	컴퓨터	3.3
김구완	53116	홍제동	정보통신	3.1
최재석	56034	양재동	정보관리	3.5
송혜영	52042	신정동	컴퓨터	2.9
조미림	53108	역삼동	정보통신	3.4

[그림 1.13] STUDENT 테이블

1.3 DBMS 발전 과정(계속)

□ 객체 지향 DBMS -> OODBMS

- ✓ 1980년대 후반 들어 새로운 데이터 모델인 객체 지향 데이터 모델이 등장
- ✓ 객체 지향 프로그래밍 패러다임을 기반으로 하는 데이터 모델
- ✓ 장점
 - 데이터와 프로그램을 그룹화하고, 복잡한 객체들을 이해하기 쉬우며, 유지와 변경이 용이함
- ✓ 예: ONTOS, OpenODB, GemStone, ObjectStore, Versant, O2 등

□ 객체 관계 DBMS

- ✓ 1990년대 후반에 관계 DBMS에 객체 지향 개념을 통합한 객체 관계 데이터 모델이 제안됨
- ✓ 예: 오라클, Informix Universal Server 등

1.4 데이터베이스 스키마/인스턴스

□ 데이터베이스 스키마(database schema)

- ✓ 전체적인 데이터베이스 구조를 뜻하며 자주 변경되지는 않음
- ✓ 데이터베이스의 모든 필요 사항을 표현할 수 있도록 설계
- ✓ 내포(intension)라고 부름

□ 데이터베이스 인스턴스(database instance(state))

- ✓ 특정 시점의 데이터베이스의 내용을 의미하며, 시간이 지남에 따라 계속해서 바뀜
- ✓ 외연(extension)이라고 부름

1.4 데이터베이스 스키마/인스턴스

데이터베이스 스키마

DEPARTMENT (DEPTNO, DEPTNAME, FLOOR)

EMPLOYEE (EMPNO, EMPNAME, TITLE, DNO, SALARY)

데이터베이스 상태

DEPARTMENT

DEPTNO	DEPTNAME	FLOOR
1	영업	8
2	기획	10
3	개발	9

DBS(i) -> DBS(i+1)

EMPLOYEE

EMPNO	EMPNAME	TITLE	DNO	SALARY
2106	김창섭	대리	2	2000000
3426	박영권	과장	3	2500000
3011	이수민	부장	1	3000000
1003	조민희	대리	1	2000000
3427	최종철	사원	3	1500000

[그림 1.1] 데이터베이스 스키마와 데이터베이스 상태

1.5 데이터베이스 언어

❑ 데이터 정의어(DDL: Data Definition Language)

- ✓ 사용자는 데이터 정의어를 사용하여 데이터베이스 스키마를 정의
- ✓ 데이터 정의어로 명시된 문장이 입력되면 DBMS는 사용자가 정의한 스키마에 대한 명세를 시스템 카탈로그 또는 데이터 사전에 저장
- ✓ 데이터 정의어의 기본적인 기능

데이터 모델에서 지원하는 데이터 구조를 생성

예, SQL에서 CREATE TABLE

데이터 구조의 변경

예, SQL에서 ALTER TABLE

데이터 구조의 삭제

예, SQL에서 DROP TABLE

데이터 접근을 위해 특정 애트리뷰트 위에 인덱스를 정의

예, SQL에서 CREATE INDEX

1.5 데이터베이스 언어

❑ 데이터 조작어(DML: Data Manipulation Language)

- ✓ 사용자는 데이터 조작어를 사용하여 데이터베이스 인스턴스 내의 원하는 데이터를 검색하고, 수정하고, 삽입하고, 삭제
- ✓ 절차적 언어(procedural language)와 비절차적 언어(non-procedural language)
- ✓ 관계 DBMS에서 사용되는 SQL은 대표적인 비절차적 언어
- ✓ 대부분의 데이터 조작어는 SUM, COUNT, AVG와 같은 내장 함수들을 갖고 있음
- ✓ 데이터 조작어는 단말기에서 대화식으로 입력되어 수행되거나 C, Java 등의 고급 프로그래밍 언어로 작성된 프로그램에 내장(embedding)되어 사용됨

1.5 데이터베이스 언어

❑ 데이터 조작용의 기본적인 기능

데이터의 검색

예, SQL에서 SELECT

데이터의 수정

예, SQL에서 UPDATE

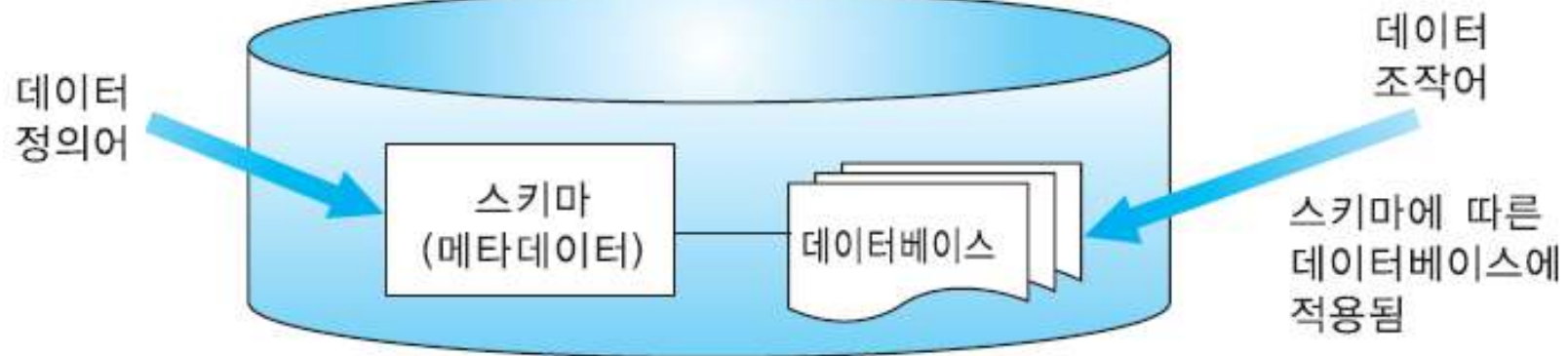
데이터의 삭제

예, SQL에서 DELETE

데이터의 삽입

예, SQL에서 INSERT

1.5 데이터베이스 언어

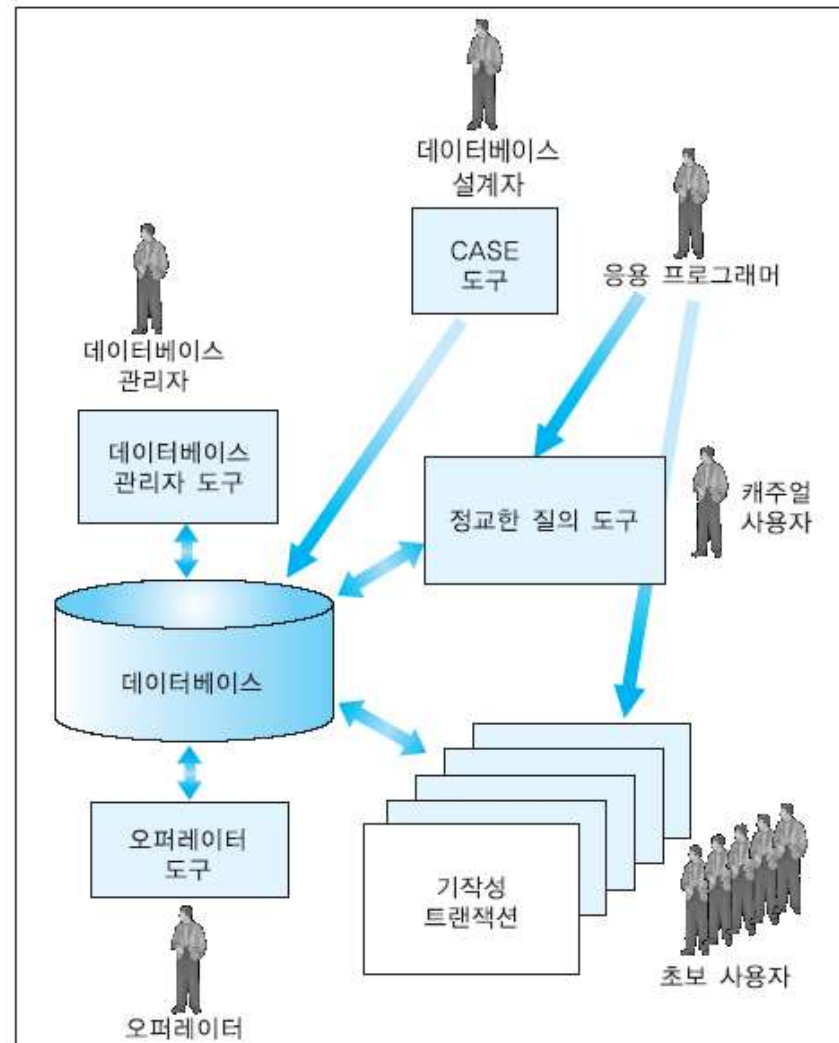


[그림 1.15] 데이터 정의어와 데이터 조작어

1.5 데이터베이스 언어

- ❑ 데이터 제어어(DCL: Data Control Language)
 - ✓ 사용자는 데이터 제어어를 사용하여 데이터베이스 트랜잭션을 명시하고 권한을 부여하거나 취소

1.6 데이터베이스 사용자



[그림 1.17] DBMS의 사용자들

1.6 데이터베이스 사용자

❑ 데이터베이스 설계자(database designer)

- ✓ ERWin 등의 CASE 도구들을 이용해서 데이터베이스 설계를 담당
- ✓ 데이터베이스의 일관성을 유지하기 위해서 정규화를 수행

❑ 응용 프로그래머

- ✓ 데이터베이스 위에서 특정 응용(예, 고객 관리, 인사 관리, 재고 관리 등)이나 인터페이스를 구현하는 사람
- ✓ 고급 프로그래밍 언어인 Java, Python 등으로 응용 프로그램을 개발하면서 데이터베이스를 접근하는 부분은 내포된 데이터 조작어를 사용
- ✓ 이들이 작성한 프로그램은 최종 사용자들이 반복해서 수행하므로 **기작성 트랜잭션**(canned transaction)이라 부름

1.6 데이터베이스 사용자

□ 데이터베이스 관리자(DBA: Database Administrator)

- ✓ 데이터베이스 관리자는 조직의 여러 부분의 상이한 요구를 만족시키기 위해서 일관성 있는 데이터베이스 스키마를 생성하고 유지하는 사람(팀)
- ✓ 데이터베이스 관리자의 역할
 - 데이터베이스 스키마의 생성과 변경
 - 무결성 제약조건을 명시
 - 사용자의 권한을 허용하거나 취소하고, 사용자의 역할을 관리
 - 저장 구조와 접근 방법(물리적 스키마) 정의
 - 백업과 회복
 - 표준화 시행

1.6 데이터베이스 사용자

□ 최종 사용자(end user)

- ✓ 질의하거나 갱신하거나 보고서를 생성하기 위해서 데이터베이스를 사용하는 사람
- ✓ 최종 사용자는 다시 데이터베이스 질의어를 사용하여 매번 다른 정보를 찾는 캐주얼 사용자와 기작성 트랜잭션을 주로 반복해서 수행하는 초보 사용자로 구분

□ 오퍼레이터

- ✓ DBMS가 운영되고 있는 컴퓨터 시스템과 전산실을 관리하는 사람

1.6 데이터베이스 사용자

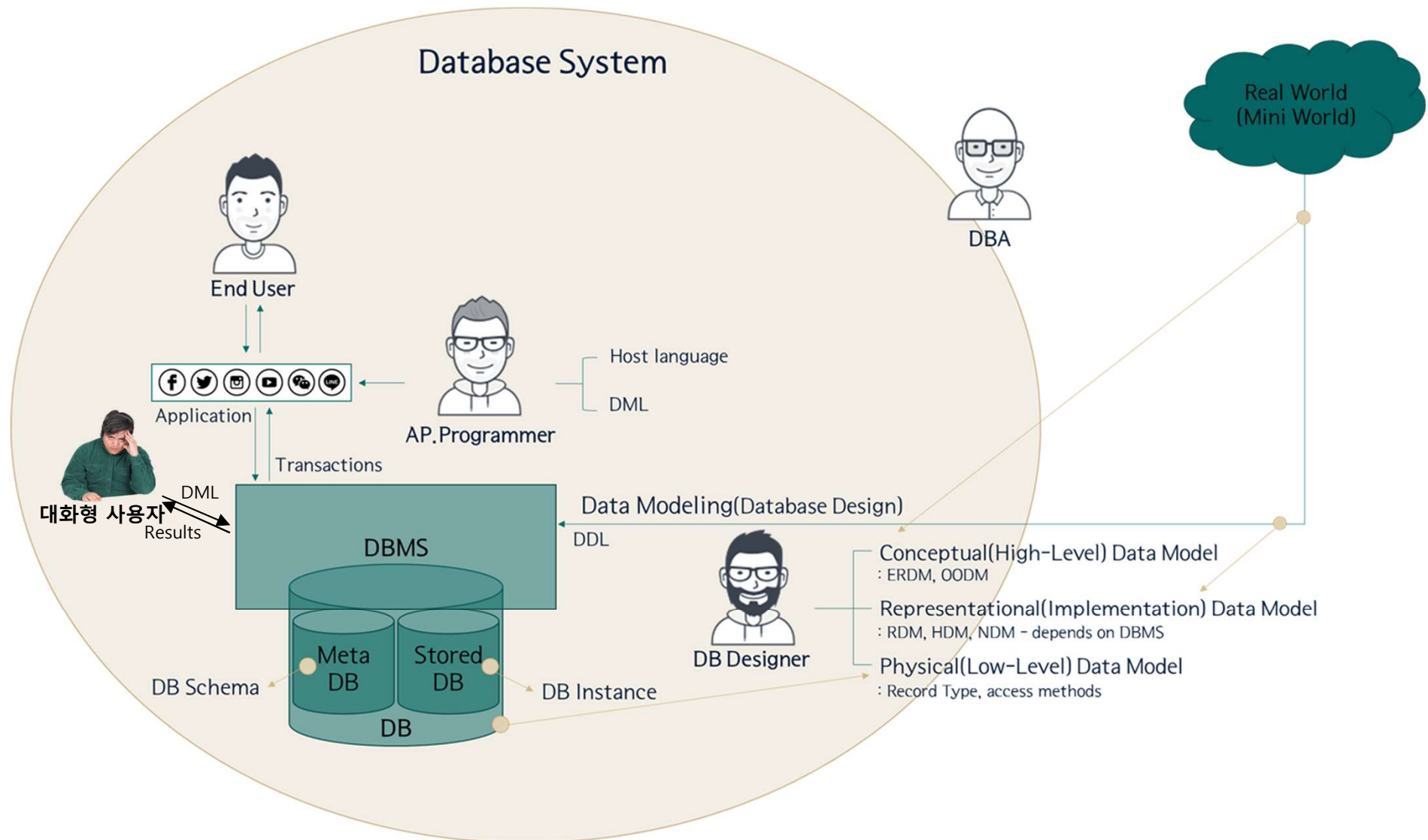


Figure 1.1-1 Database users for database system.

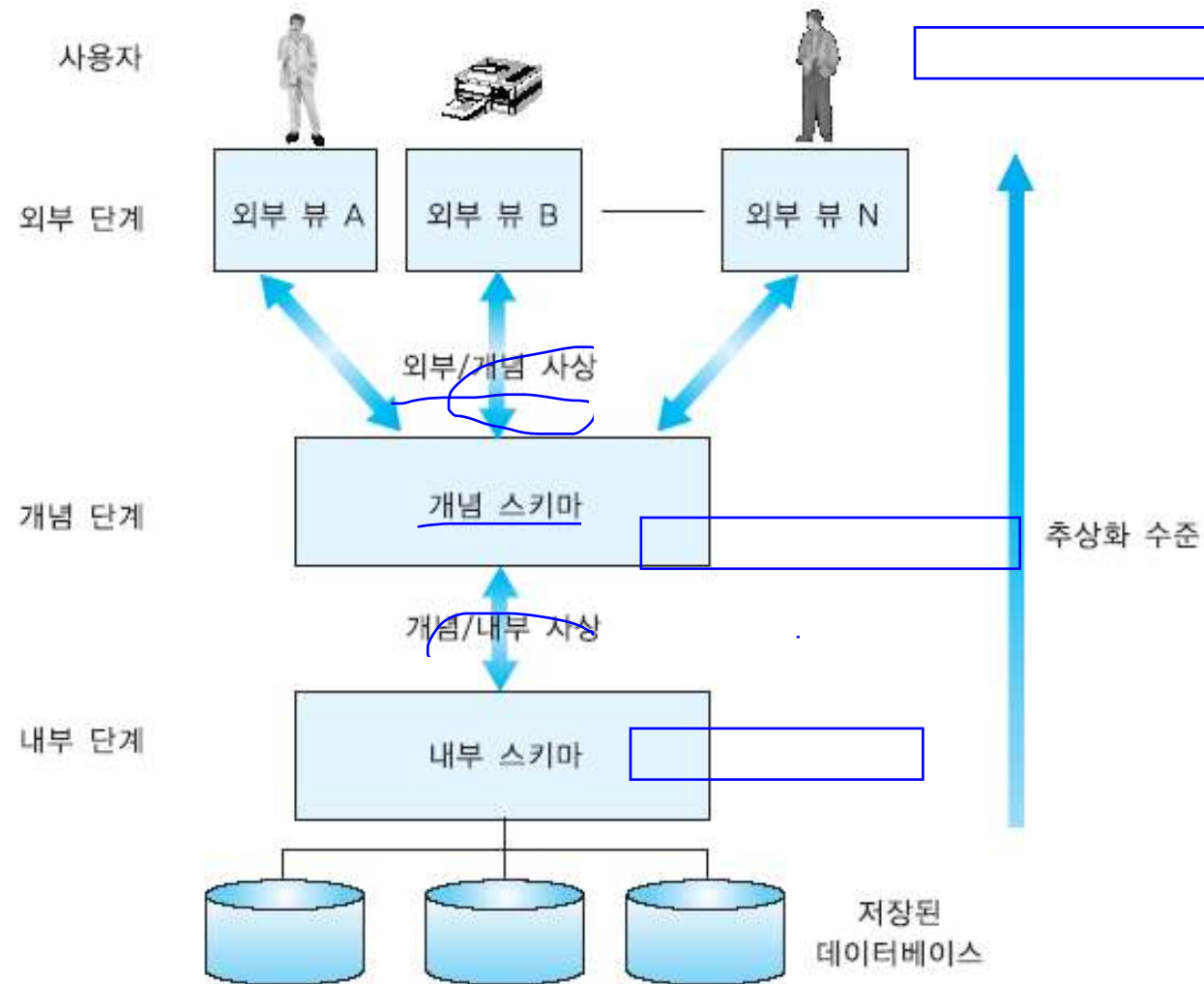
1.7 ANSI/SPARC 아키텍처와 데이터 독립성

□ ANSI/SPARC 아키텍처

- ✓ 현재의 대부분의 상용 DBMS 구현에서 사용되는 일반적인 아키텍처는 1978년에 제안된 ANSI/SPARC 아키텍처
- ✓ ANSI/SPARC 아키텍처의 3단계는 물리적, 개념적, 외부 단계로 이루어짐
 - 외부 단계(external level): 각 사용자의 뷰
 - 개념 단계(conceptual level): 사용자 공동체의 뷰
 - 내부 단계(internal level): 물리적 또는 저장 뷰



1.7 ANSI/SPARC 아키텍처와 데이터 독립성



[그림 1.18] ANSI/SPARC 3단계 아키텍처

1.7 ANSI/SPARC 아키텍처와 데이터 독립성

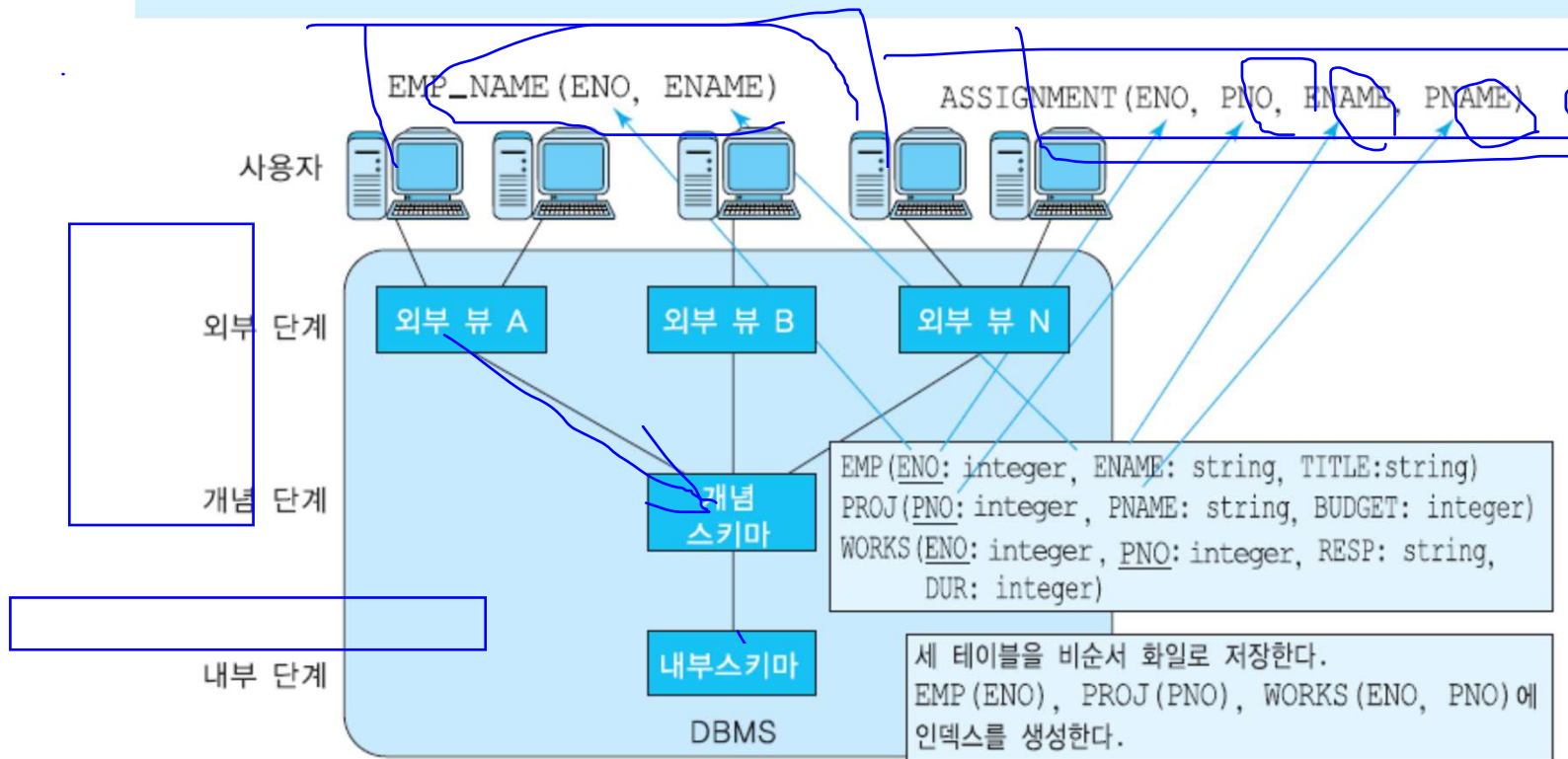
□ 외부 단계

- ✓ 일반적으로, 최종 사용자와 응용 프로그래머들은 데이터베이스의 일부분에만 관심을 가짐
- ✓ 데이터베이스의 각 사용자가 갖는 뷰
- ✓ 여러 부류의 사용자를 위해 동일한 개념 단계로부터 다수의 서로 다른 뷰가 제공될 수 있음

1.7 ANSI/SPARC 아키텍처와 데이터 독립성

예 :

그림 1.19는 관계 데이터 모델에서 개념 단계의 세 테이블 EMP, PROJ, WORKS로부터 일부 애틀리뷰트들을 선택하여 외부 단계의 뷰 ASSIGNMENT와 EMP_NAME이 정의된 예를 보여준다.



[그림 1.19] 관계 데이터 모델에서 3단계 아키텍처의 예

1.7 ANSI/SPARC 아키텍처와 데이터 독립성

□ 개념 단계

- ✓ 조직체의 정보 모델로서, 물리적인 구현은 고려하지 않으면서 조직체 전체에 관한 스키마를 포함
- ✓ 데이터베이스에 어떤 데이터가 저장되어 있으며, 데이터 간에는 어떤 관계가 존재하고, 어떤 무결성 제약조건들이 명시되어 있는가를 기술함
- ✓ 데이터베이스에 대한 사용자 공동체의 뷰를 나타냄
- ✓ 데이터베이스마다 오직 한 개의 개념 스키마가 존재

예 :

그림 1.19에서는 개념 단계가 세 테이블 EMP, PROJ, WORKS로 구성되어 있다.

1.7 ANSI/SPARC 아키텍처와 데이터 독립성

□ 내부 단계

- ✓ 실제의 물리적인 데이터 구조에 관한 스키마
- ✓ 데이터베이스에 어떤 데이터가 어떻게 저장되어 있는가를 기술함
- ✓ 인덱스, 해싱 등과 같은 접근 경로, 데이터 압축 등을 기술함
- ✓ 데이터베이스의 개념 스키마에는 영향을 미치지 않으면서 성능을 향상시키기 위해 내부 스키마를 변경하는 것이 바람직
- ✓ 내부 단계 아래는 물리적 단계
- ✓ 물리적 단계는 DBMS의 지시에 따라 운영 체제가 관리함

1.7 ANSI/SPARC 아키텍처와 데이터 독립성

□ 스키마 간의 사상

- ✓ DBMS는 세 가지 유형의 스키마 간의 사상을 책임짐
- ✓ 외부/개념 사상(external/conceptual mapping)
 - 외부 단계의 뷰를 사용해서 입력된 사용자의 질의를 개념 단계의 스키마를 사용한 질의로 변환
- ✓ 개념/내부 사상(conceptual/internal mapping)
 - 이를 다시 내부 단계의 스키마로 변환하여 디스크의 데이터베이스를 접근

1.7 ANSI/SPARC 아키텍처와 데이터 독립성

□ 데이터 독립성

✓ 상위 단계의 스키마 정의에 영향을 주지 않으면서 어떤 단계의 스키마 정의를 변경할 수 있음을 의미

■ 물리적인 데이터 독립성(physical data independence)

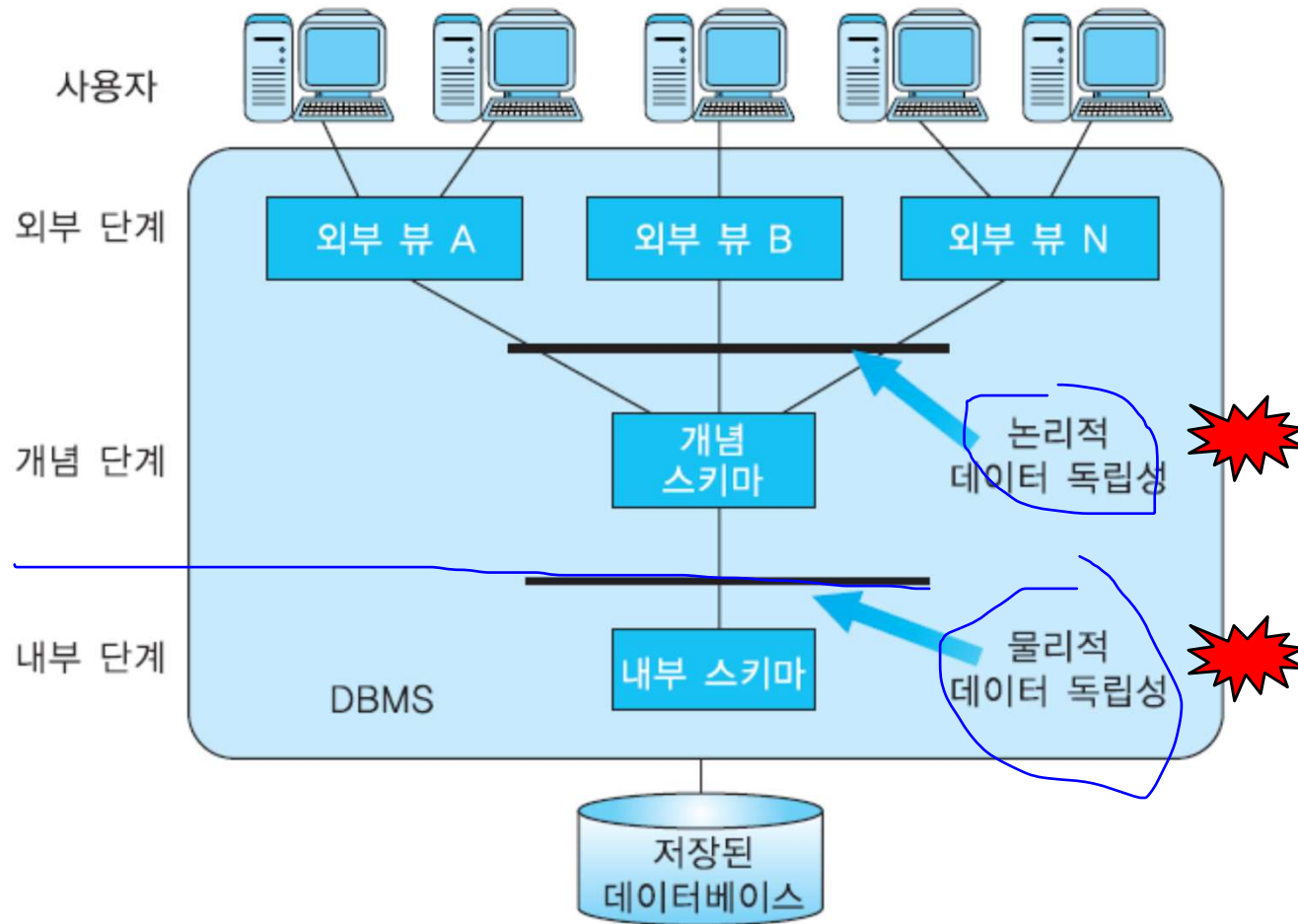
✓ 내부 스키마의 변화가 개념적 스키마에 영향을 미치지 않으며, 따라서 외부 스키마(또는 응용 프로그램)에도 영향을 미치지 않음을 의미

■ 논리적인 데이터 독립성(logical data independence)

✓ 개념 스키마의 변화로부터 외부 스키마가 영향을 받지 않음을 의미

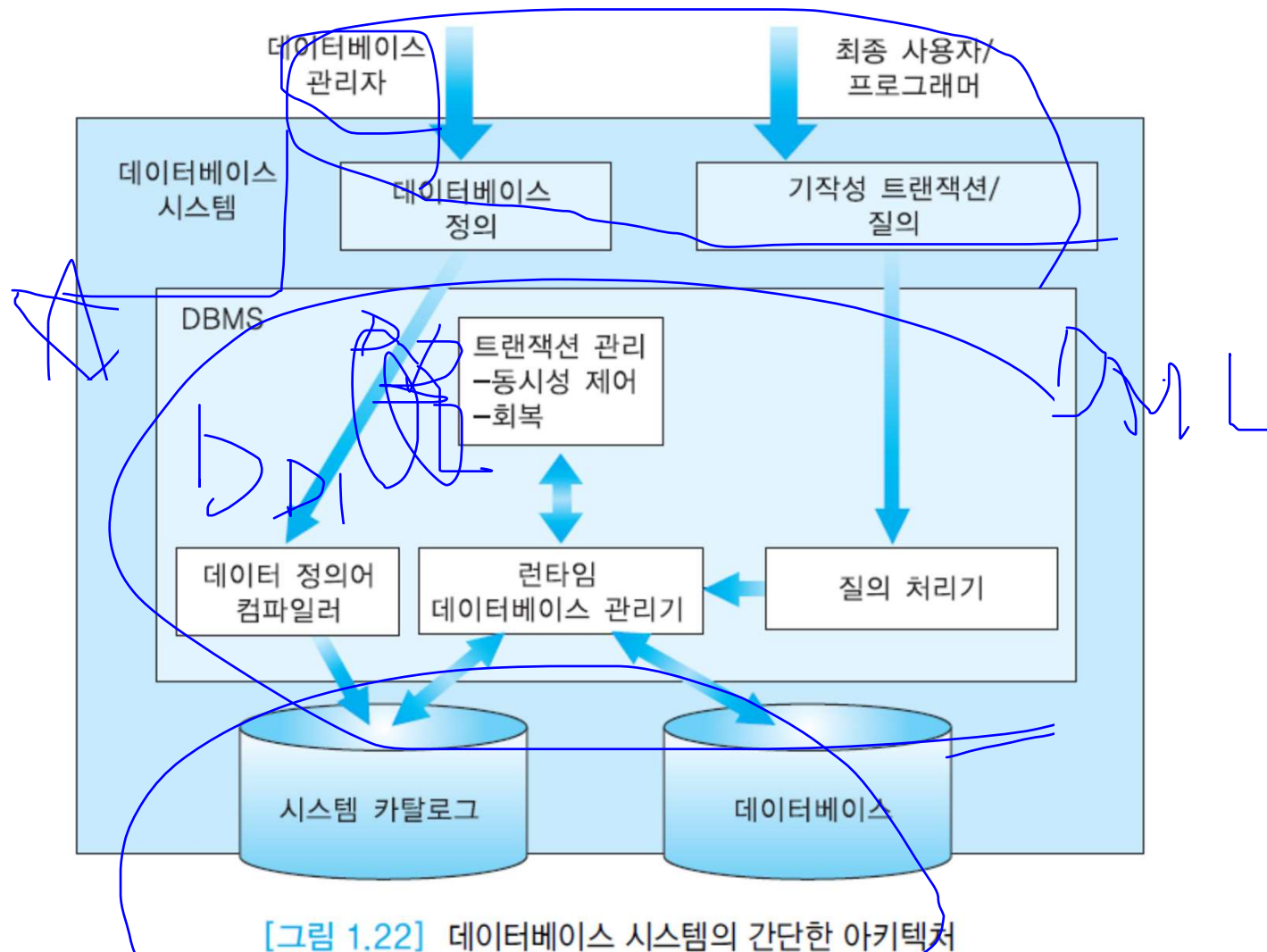
✓ 기존의 외부 스키마에 영향을 미치지 않고, 응용 프로그램을 다시 작성할 필요 없이 개념 스키마에 대한 변화가 가능해야 함

1.7 ANSI/SPARC 아키텍처와 데이터 독립성



[그림 1.21] 논리적 데이터 독립성과 물리적 데이터 독립성

1.8 데이터베이스 시스템 아키텍처



1.8 데이터베이스 시스템 아키텍처

- ❑ 데이터 정의어 컴파일러(DDL compiler) 모듈
 - ✓ 데이터 정의를 사용하여 테이블 생성을 요청하면 테이블을 파일 형태로 데이터베이스에 만들고, 이 테이블에 대한 명세를 시스템 카탈로그에 저장
- ❑ 질의 처리기(query processor) 모듈
 - ✓ 데이터 조작어를 수행하는 최적의 방법을 찾는 모듈을 통해서 기계어 코드로 번역
- ❑ 런타임 데이터베이스 관리기(run-time database manager) 모듈
 - ✓ 디스크에 저장된 데이터베이스를 접근
- ❑ 트랜잭션 관리(transaction management) 모듈
 - ✓ 동시성 제어(concurrency control) 모듈
 - ✓ 회복(recovery) 모듈

DBMS